

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf\_095035a0-e9f\agent-aabb69b02bf1328ff.jsonl`
- SHA-256: `10eb1653d9e01fd7978fd33e7592ded9ee86629873346cd3e7f6fd1ce9fa95a8`
- Source modified: `2026-06-13T19:16:51+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf\_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:15:05.361Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"frame_height has no fallback (unlike frame_width); fh=0 silently collapses person features to ~0 ? untested asymmetry","severity":"high","file":"backend/pipeline/segmenters/fusion_hybrid.py:99-100","detail":"`fw = det[\"frame\_width\"]` or `frame\_width` falls back to the substrate's frame_width when cv2 reports 0, but `fh = det[\"frame\_height\"]` or `0.0` falls back to 0.0 ? there is no analogous recovery. When a codec causes CAP_PROP_FRAME_HEIGHT to read 0 (the detections_per_frame docstring itself warns cv2 props are flaky per-codec), `\_foot\_norm/\_center\_norm` return (0,0) for every box, so mean_player_motion, in_court_ratio, two_sided_motion, players_in_court (under any polygon) and shuttle_player_colocation all collapse to ~0 silently ? i.e. every gated window looks like dead time. I confirmed this: with fh=0 the aggregate returns mean_player_motion=0.0 and shuttle_player_colocation=0.0 vs 0.002 and 1.0 with fh=1000. No test exercises a zero/missing frame_height, so this asymmetry would ship unnoticed. The person-cue test hard-codes frame_height=1000.0, dodging it.", "fix":"Either give frame_height a sane fallback symmetric with frame_width (e.g. derive from substrate aspect, or carry frame_height through _probe), or have compute_person_features short-circuit/raise on non-positive dims. Add a test feeding det[\"frame_height\"]=0.0 and asserting the chosen behavior (fallback used, or features explicitly flagged unavailable rather than silently zeroed)."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:09.079Z)

I'll verify this finding by reading the actual code. Let me start by examining the key files.

3. user (2026-06-13T19:15:10.037Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
```

```

4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of **scale/translation-invariant**
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in **pixels**; ``(track_id, box)`` when motion
    needs it.
12    - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
    frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
    timing).
14    - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
    trajectory shape.
15    - ``court_polygon`` is normalised 0-1 ``[(x, y), ...]`` or ``None`` (None ? no mask,
    every
16        detection counts ? the player-count-only mode).
17    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
    0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)
38        j = n - 1
39        for i in range(n):
40            xi, yi = polygon[i]
41            xj, yj = polygon[j]
42            # Does the horizontal ray at y cross edge (i, j)?
43            if (yi > y) != (yj > y):
44                x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                if x < x_cross:
46                    inside = not inside
47            j = i
48        return inside
49
50
51    def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
    float]:
52        """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53        the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54        if frame_width <= 0 or frame_height <= 0:
55            return (0.0, 0.0)
56        x1, _y1, x2, y2 = box
57        return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60    def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,

```

```

float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                         court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                        frame_height: float,
78                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
82 court_polygon))
83             for _frame_idx, dets in sorted(per_frame.items())
84         ]
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)
91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:

```

```

122         return 0.0
123     return sum(steps) / len(steps)
124
125
126 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                     frame_height: float, net: Optional[Net],
128                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133     0.0.
134     """
135     if net is None or not per_frame:
136         return 0.0
137     axis, pos = net
138     both = 0
139     for _frame_idx, dets in per_frame.items():
140         near, far = False, False
141         for _tid, box in dets:
142             if not person_in_court(box, frame_width, frame_height, court_polygon):
143                 continue
144             fx, fy = _foot_norm(box, frame_width, frame_height)
145             coord = fx if axis == "x" else fy
146             if coord < pos:
147                 near = True
148             else:
149                 far = True
150             if near and far:
151                 both += 1
152     return both / len(per_frame)
153
154 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                    pad_frac: float) -> bool:
156     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157     width/height (the 'adjacent / in-hand' tolerance)?"""
158     x1, y1, x2, y2 = box
159     px = pad_frac * frame_width
160     py = pad_frac * frame_height
161     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                               frame_width: float, frame_height: float,
166                               pad_frac: float = 0.03) -> float:
167     """Fraction of (visible shuttle frames that were also person-sampled) in which the
168     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
169     ~0**
170     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
171     """
172     if not per_frame or frame_width <= 0 or frame_height <= 0:
173         return 0.0
174     aligned = 0
175     colocated = 0
176     for p in shuttle_points or []:
177         if not getattr(p, "visible", False):
178             continue
179         dets = per_frame.get(int(p.frame))
180         if dets is None: # shuttle frame wasn't person-sampled ? can't judge
181             continue
182         aligned += 1
183         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)

```

```

183         for _tid, box in dets):
184             colocated += 1
185     return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                 frame_width: float, frame_height: float,
198                                 court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                 net: Optional[Net] = None,
200                                 min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212

```

4. user (2026-06-13T19:15:10.560Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13    never imports torch / touches the GPU.
14    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17    **Persisted candidates remain the full superset** regardless, so the offline
experiment
18    harness can re-score any variant.
19
20    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default

```

```

23     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24     experiment harness (EXPERIMENT_HARNESS.md), never silently.
25     """
26     from typing import Any, Dict, List, Optional, Tuple
27
28     from backend.pipeline.segmenters.base import segmenter_registry
29     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30     from backend.pipeline.detectors.base import DetectorRunner
31     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33     from backend.pipeline.detectors import rally_gate
34     from backend.pipeline.detectors import fusion_features as ff
35
36
37     class FusionSegmenter(TrajectoryHybridSegmenter):
38         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40         family = "fusion"
41         display_label = "Fusion"
42
43         def __init__(self, db: Any, config: Any):
44             super().__init__(db, config)
45             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46             self._person: Optional[Any] = None
47
48         @property
49         def name(self) -> str:
50             return "fusion_hybrid"
51
52         @property
53         def description(self) -> str:
54             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                     "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                     "AI provider sets semantic boundaries.")
57
58         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60             if substrate == "tracknet":
61                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62                 return TrackNetRunner(cfg), {
63                     "weights_path": cfg.weights_path,
64                     "queue_length": cfg.queue_length,
65                     "fusion_substrate": "tracknet",
66                 }
67             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                     frame_width: float, video_path: str,
72                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76             # over the whole trajectory by rally_gate (camera-agnostic).
77             gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,

```

```

min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                         frame_width: float, video_path: str,
87                         fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109    def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                           window_stats: List[Dict[str, Any]],
111                           idx_cfg: Dict[str, Any]) -> List[int]:
112        """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113        (0) keep every window ? identical to the substrate. Persisted candidates are
114        untouched (full superset)."""
115        fcfg = idx_cfg.get("fusion", {})
116        min_crossings = int(fcfg.get("min_crossings", 0))
117        min_players = int(fcfg.get("min_players", 0))
118        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120        keep: List[int] = []
121        for i, st in enumerate(window_stats):
122            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                continue
124            if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                continue
126            keep.append(i)
127        return keep
128
129
130    segmenter_registry.register(FusionSegmenter)
131

```

Now let me verify the claims. I need to check the `person\_detector.py` to see how `detections\_per\_frame` is structured and whether `frame\_height` could be 0, and look at the test that the finding mentions.

6. user (2026-06-13T19:15:15.672Z)

```
1      """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3      Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4      the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5      `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
6      identically);
7      next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
8      later the learned classifier reads its persisted features. Keeping detection here (not
9      in
10     the segmenter) is what makes "add a cue = register a producer" true.
11
12     Behaviour is unchanged from the in-lined version ? only the home moved:
13     - `lazy_load_model`           ? load a permissively-licensed torchvision detector
14     - `make_tracker`             ? a fresh supervision ByteTrack per chunk
15     - `detect_and_track`         ? one frame ? confident persons with persistent IDs
16     - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
17     with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
18     that feed the candidate `stats` JSON and the learned fusion features.
19
20     Heavy deps (torchvision, supervision) are imported lazily inside the methods so
21     importing
22     the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
23     the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
24     (ADR-005; ultralytics' AGPL YOLO was dropped).
25     """
26     import logging
27     from typing import Any, Dict, List, Optional, Tuple
28
29     import cv2
30     import torch
31
32     from backend.utils.geometry import get_velocity_normalised
33
34     logger = logging.getLogger(__name__)
35
36     # torchvision detection models are trained on COCO where the "person" category is
37     # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
38     # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
39     _PERSON_CLASS_ID = 1
40
41     # Permissively-licensed torchvision detectors selectable via config
42     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
43     _DETECTOR_FACTORIES = {
44         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
45         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
46         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
47     }
48     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
49
50     class PersonDetector:
51         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55         min_action_window_duration}``); the model loads lazily on first use.
56         """
57
58         def __init__(self, config: Dict[str, Any]):
```



```

57         self.config = config
58         # Dynamic torchvision module (None until lazy_load_model); typed Any so the
59         # `self.model([tensor])` inference call type-checks without a quarantine.
60         self.model: Any = None
61         self.device: Optional[str] = None
62
63     def lazy_load_model(self) -> None:
64         if self.model is not None:
65             return
66
67         # Imported lazily so the module imports without torchvision present and so
68         # the test suite (which pre-sets self.model) never pulls in model weights.
69         from torchvision.models import detection as tv_detection
70
71         idx_cfg = self.config.get("indexing", {})
72         use_gpu = idx_cfg.get("use_gpu", True)
73         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74         if use_gpu and not torch.cuda.is_available():
75             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78         if model_name not in _DETECTOR_FACTORIES:
79             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80             model_name = _DEFAULT_DETECTOR
81
82         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85         self.model = builder(weights="DEFAULT")
86         self.model.eval()
87         self.model.to(self.device)
88
89     def make_tracker(self, fps: float) -> Any:
90         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92         Imported lazily so tests can mock this method without supervision installed.
93         """
94         import supervision as sv
95         frame_rate = max(1, int(round(fps)))
96         return sv.ByteTrack(frame_rate=frame_rate)
97
98     def detect_and_track(self, frame: Any, tracker: Any,
99                         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100         """Detect persons in a frame and assign persistent track IDs.
101
102         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103         This replaces the old ultralytics ``model.track(...)`` call, which bundled
104         detection AND association: torchvision does the detection, supervision's
105         ByteTrack does the association.
106         """
107         import supervision as sv
108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
112         1).float().div(255.0).to(self.device)
113         with torch.no_grad():
114             outputs = self.model([tensor])
115
116         out = outputs[0]
117         boxes = out["boxes"].detach().cpu().numpy()
118         labels = out["labels"].detach().cpu().numpy()

```

```

118         scores = out["scores"].detach().cpu().numpy()
119
120         # Keep only confident person detections.
121         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
122         if not mask.any():
123             return []
124
125         # ByteTrack requires confidence to be populated (it filters on it internally).
126         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
127         tracked = tracker.update_with_detections(dets)
128
129         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130         for i in range(len(tracked)):
131             tid = tracked.tracker_id[i]
132             if tid is None:
133                 continue
134             x1, y1, x2, y2 = tracked.xyxy[i]
135             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136         return result
137
138     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139                                         velocity_thresh: float, merge_gap: float,
140                                         frame_skip: int = 3,
141                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142         """
143         Runs person detection + tracking on a video chunk and extracts high-motion
144         action windows. Returns a list of dicts in the full video's timeframe, each
with:
145             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146             track_id_count, chunk_index.
147
148         Args:
149             frame_skip: Process every Nth frame (1 = every frame). Higher values
150                 dramatically reduce inference cost with minimal quality loss.
151             chunk_index: Index of the source chunk (recorded on each window for
traceability).
152         """
153         cap = cv2.VideoCapture(chunk_path)
154         if not cap.isOpened():
155             logger.error(f"Could not open {chunk_path}")
156             return []
157
158         fps = cap.get(cv2.CAP_PROP_FPS)
159         if fps <= 0:
160             fps = 30.0
161
162         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163         if frame_width <= 0:
164             frame_width = 1280.0 # Sensible fallback
165
166         # Effective sampling rate after frame-skipping
167         effective_fps = fps / max(frame_skip, 1)
168
169         # A fresh tracker per chunk ? track IDs only need to be consistent within a
170         # chunk (windows are computed per chunk, then merged across chunks by time).
171         tracker = self.make_tracker(effective_fps)
172         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175         # so windows can be enriched with motion stats for logging + fine-tuning.
176         active_frames = []
177         frame_idx = 0

```

```

178     track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180     while True:
181         ret, frame = cap.read()
182         if not ret:
183             break
184
185         # Skip frames for performance ? at 30fps, every 3rd frame still
186         # gives ~10 samples/second, plenty for human-scale velocity.
187         if frame_idx % max(frame_skip, 1) != 0:
188             frame_idx += 1
189             continue
190
191         # Detect persons + assign persistent track IDs.
192         detections = self.detect_and_track(frame, tracker, score_thresh)
193
194         frame_peak_velocity = 0.0
195         frame_active_ids = set()
196         current_ids = set()
197
198         for track_id, bbox in detections:
199             current_ids.add(track_id)
200
201             if track_id in track_history:
202                 prev_bbox = track_history[track_id]
203                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204
205                 if velocity > velocity_thresh:
206                     frame_peak_velocity = max(frame_peak_velocity, velocity)
207                     frame_active_ids.add(track_id)
208
209                 track_history[track_id] = bbox
210
211         # Prune stale IDs that are no longer visible ? prevents false
212         # velocity spikes when the tracker reassigns an old ID.
213         track_history = {k: v for k, v in track_history.items() if k in current_ids}
214
215         if frame_active_ids:
216             active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
217
218         frame_idx += 1
219
220     cap.release()
221
222     # Group active frames into continuous windows
223     if not active_frames:
224         return []
225
226     max_gap_frames = int(merge_gap * fps)
227
228     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
229         velocities = [v for _, v, _ in group]
230         track_ids = set().union(*[ids for _, _, ids in group])
231         return {
232             "start": group[0][0] / fps + chunk_start,
233             "end": group[-1][0] / fps + chunk_start,
234             "active_frame_count": len(group),
235             "peak_velocity": max(velocities),
236             "mean_velocity": sum(velocities) / len(velocities),
237             "track_id_count": len(track_ids),
238             "chunk_index": chunk_index,
239         }
240
241     windows = []
242     group = [active_frames[0]]

```

```

242         for af in active_frames[1:]:
243             if af[0] - group[-1][0] <= max_gap_frames:
244                 group.append(af)
245             else:
246                 windows.append(_finalize(group))
247                 group = [af]
248         windows.append(_finalize(group))
249
250         # Filter out extremely short action windows
251         min_window_duration = self.config.get("indexing",
252         {}).get("min_action_window_duration", 2.0)
253         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
254         return windows
255
256     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257                             frame_skip: int = 3) -> Dict[str, Any]:
258         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
259         video.
260         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
261         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
262         discards the
263         per-frame boxes), this returns the raw per-frame detections so the fusion
264         features can
265         align them with the shuttle trajectory. It reads the source by **absolute frame
266         index**
267         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
268         match
269         the WASB trajectory's ``.frame`` exactly ? that alignment is what
270         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
271         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
272         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
273         seek
274         accuracy varies by codec (it lands near the requested keyframe then decodes
275         forward);
276         treat the indices as ±a few frames on real footage ? fine for window-level
277         features.
278         """
279         self.lazy_load_model()
280         cap = cv2.VideoCapture(video_path)
281         if not cap.isOpened():
282             logger.error(f"Could not open {video_path}")
283             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
284
285         fps = cap.get(cv2.CAP_PROP_FPS)
286         if fps <= 0:
287             fps = 30.0
288         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
289         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
290         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
291         0.5)
292         step = max(frame_skip, 1)
293         tracker = self.make_tracker(fps / step)
294
295         start_frame = max(0, int(start_frame))
296         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
297         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
298         frame_idx = start_frame
299         while frame_idx <= end_frame:
300             ret, frame = cap.read()
301             if not ret:
302                 break
303             if (frame_idx - start_frame) % step == 0:

```

```

297         frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
298         frame_idx += 1
299         cap.release()
300         return {"frames": frames, "fps": fps,
301               "frame_width": float(frame_width), "frame_height": float(frame_height)}
302

```

7. user (2026-06-13T19:15:15.930Z)

```

diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

    return windows

+
+ def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                           frame_skip: int = 3) -> Dict[str, Any]:
+     """Per-frame person detections over `[start_frame, end_frame]` of the ORIGINAL video.
+
+     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+     per-frame boxes), this returns the raw per-frame detections so the fusion features can
+     align them with the shuttle trajectory. It reads the source by **absolute frame index**
+     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+     the WASB trajectory's ``.frame`` exactly ? that alignment is what
+     ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
+     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+     accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+     treat the indices as ±a few frames on real footage ? fine for window-level features.
+     """
+     self.lazy_load_model()
+     cap = cv2.VideoCapture(video_path)
+     if not cap.isOpened():
+         logger.error(f"Could not open {video_path}")
+         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+     fps = cap.get(cv2.CAP_PROP_FPS)
+     if fps <= 0:
+         fps = 30.0
+     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+     step = max(frame_skip, 1)
+     tracker = self.make_tracker(fps / step)
+
+     start_frame = max(0, int(start_frame))
+     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+     frame_idx = start_frame
+     while frame_idx <= end_frame:
+         ret, frame = cap.read()
+         if not ret:
+             break
+         if (frame_idx - start_frame) % step == 0:
+             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+             frame_idx += 1
+     cap.release()
+     return {"frames": frames, "fps": fps,
+           "frame_width": float(frame_width), "frame_height": float(frame_height)}

```

8. assistant (2026-06-13T19:15:21.301Z)

Now I've confirmed the core of the finding. Let me verify the asymmetry claims precisely:

1. Line 99: `fw = det["frame\_width"] or frame\_width` ? falls back to the substrate `frame\_width` (a real non-zero value passed in).
2. Line 100: `fh = det["frame\_height"] or 0.0` ? falls back to `0.0`.

And in `person\_detector.detections\_per\_frame`, unlike `extract\_action\_windows\_from\_chunk` (line 162-164 which has a `frame\_width <= 0` ? 1280.0 fallback), there is NO fallback for either `frame\_width` OR `frame\_height` ? they're read raw from cv2 props (lines 282-283).

Now let me verify the docstring warning about cv2 props being flaky per-codec, and check the tests to confirm no test exercises zero/missing frame_height.

9. user (2026-06-13T19:15:22.420Z)

```
backend\pipeline\detectors\fusion_features.py:51: def _foot_norm(box: BBox, frame_width: float,
frame_height: float) -> Tuple[float, float]:
backend\pipeline\detectors\fusion_features.py:54:     if frame_width <= 0 or frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:57:     return ((x1 + x2) / 2.0 / frame_width, y2 /
frame_height)
backend\pipeline\detectors\fusion_features.py:60: def _center_norm(box: BBox, frame_width: float,
frame_height: float) -> Tuple[float, float]:
backend\pipeline\detectors\fusion_features.py:61:     if frame_width <= 0 or frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:64:     return ((x1 + x2) / 2.0 / frame_width, (y1
+ y2) / 2.0 / frame_height)
backend\pipeline\detectors\fusion_features.py:67: def person_in_court(box: BBox, frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:73:     return point_in_polygon(_foot_norm(box,
frame_width, frame_height), court_polygon)
backend\pipeline\detectors\fusion_features.py:77:                                     frame_height: float,
backend\pipeline\detectors\fusion_features.py:81:                                     sum(1 for _tid, box in dets if
person_in_court(box, frame_width, frame_height, court_polygon))
backend\pipeline\detectors\fusion_features.py:107:                                     frame_width: float,
frame_height: float) -> float:
backend\pipeline\detectors\fusion_features.py:118:                                     cx0, cy0 =
_center_norm(prev[tid], frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:119:                                     cx1, cy1 = _center_norm(box,
frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:127:                                     frame_height: float, net:
Optional[Net],
backend\pipeline\detectors\fusion_features.py:141:                                     if not person_in_court(box,
frame_width, frame_height, court_polygon):
backend\pipeline\detectors\fusion_features.py:143:                                     fx, fy = _foot_norm(box,
frame_width, frame_height)
backend\pipeline\detectors\fusion_features.py:154: def _shuttle_in_box(sx: float, sy: float, box:
BBox, frame_width: float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:160:     py = pad_frac * frame_height
backend\pipeline\detectors\fusion_features.py:165:                                     frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:171:     if not per_frame or frame_width <= 0 or
frame_height <= 0:
backend\pipeline\detectors\fusion_features.py:182:     if any(_shuttle_in_box(p.x, p.y, box,
frame_width, frame_height, pad_frac)
backend\pipeline\detectors\fusion_features.py:197:                                     frame_width:
float, frame_height: float,
backend\pipeline\detectors\fusion_features.py:203:     counts = in_court_counts(per_frame,
frame_width, frame_height, court_polygon)
backend\pipeline\detectors\fusion_features.py:206:     "two_sided_motion":
two_sided_motion(per_frame, frame_width, frame_height, net, court_polygon),
backend\pipeline\detectors\fusion_features.py:207:     "mean_player_motion":
mean_player_motion(per_frame, frame_width, frame_height),
backend\pipeline\detectors\fusion_features.py:210:     per_frame, shuttle_points,
frame_width, frame_height),
```

```

backend\pipeline\detectors\person_detector.py:269:         "frame_height"}}`. A fresh window-
local ByteTrack tracker is used. ? cv2 frame-peek
backend\pipeline\detectors\person_detector.py:277:         return {"frames": {}, "fps": 30.0,
"frame_width": 0.0, "frame_height": 0.0}
backend\pipeline\detectors\person_detector.py:283:         frame_height =
cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
backend\pipeline\detectors\person_detector.py:301:         "frame_width":
float(frame_width), "frame_height": float(frame_height))}
backend\pipeline\segmenters\fusion_hybrid.py:100:         fh = det["frame_height"] or 0.0
tests\test_fusion_hybrid.py:102:         "fps": 30.0, "frame_width": 1000.0, "frame_height":
1000.0,

```

10. user (2026-06-13T19:15:22.745Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajjectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be replaced by CRLF the next time Git touches it

```

backend/config/models.py | 34 ++++++
backend/pipeline/detectors/person_detector.py | 47 ++++++
backend/pipeline/segmenters/__init__.py | 2 +
backend/pipeline/segmenters/trajjectory_hybrid.py | 50 ++++++-----
docs/README.md | 3 +-
tests/test_config.py | 30 ++++++
tests/test_trajectory_hybrid_equivalence.py | 22 ++++++
7 files changed, 177 insertions(+), 11 deletions(-)

```

11. assistant (2026-06-13T19:15:26.126Z)

The test at `tests/test\_fusion\_hybrid.py:102` hard-codes `frame\_height: 1000.0`, confirming the finding's claim that the person-cue test dodges the zero case. Let me look at that test file to confirm what it asserts and whether any test exercises a zero frame_height. Let me also verify the docstring warning the finding references about cv2 props being flaky per-codec.

12. user (2026-06-13T19:15:26.756Z)

```

1      """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3      Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4      keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5      substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6      persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7      """
8      from types import SimpleNamespace
9      from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17              "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19              "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22      def _points():
23          pts = []
24          for f in range(30, 120):          # window A frames: y flips 100<->900 around the net
(~500)

```

```

25         pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26     for f in range(300, 360):        # window B frames: y == net level -> deadband -> no
crossings
27         pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28     return pts
29
30
31     def _runner():
32         r = MagicMock()
33         r.healthcheck.return_value = (True, "env ok")
34         r.run_predict.return_value = "out.csv"
35         r.parse_trajectory_csv.return_value = _points()
36         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37         return r
38
39
40     def _run(indexing=None, provider_segments=None):
41         provider = MagicMock()
42         provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
43         db = MagicMock()
44         db.add_candidate_segment.side_effect = [101, 102, 103, 104]
45         db.add_play_segment.return_value = 200
46
47         with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
48             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
49             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
50             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
52             patch("os.environ.get", return_value="dummy_key"):
53             preg.get.return_value = provider
54             seg = FusionSegmenter(db, {"indexing": indexing or {}})
55             res = seg.process_video("v1", "clip.mp4")
56             return db, provider, res
57
58
59     def _stats_calls(db):
60         """Persisted stats dicts, in call order."""
61         return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
62
63
64     def test_net_crossings_persisted_and_person_off_by_default():
65         db, _provider, _res = _run()
66         stats = _stats_calls(db)
67         assert len(stats) == 2                                # both windows persisted
(superset)
68         assert all("net_crossings" in s for s in stats)
69         # Window A (rally) crosses many times; window B (held) zero.
70         assert stats[0]["net_crossings"] >= 2
71         assert stats[1]["net_crossings"] == 0.0
72         # Person cue OFF by default -> NONE of the person features present.
73         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
74         # The 4 base motion keys are still there.
75         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
76                     for s in stats)
77
78
79     def test_persisted_candidates_are_superset_of_substrate_windows():

```



```

80         db, _provider, _res = _run()
81         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82         assert spans == [(1.0, 4.0), (10.0, 12.0)]           # exactly the substrate windows,
unfiltered
83
84
85     def test_default_hands_off_all_windows():
86         _db, provider, _res = _run()                         # min_crossings default 0 -> no
gating
87         assert provider.analyze_video.call_count == 2
88
89
90     def test_gate_a_drops_low_crossing_window_from_handoff_only():
91         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92         # Persistence is still the full superset (offline substrate intact)...
93         assert len(_stats_calls(db)) == 2
94         # ...but only the high-crossing window A reaches the AI handoff.
95         assert provider.analyze_video.call_count == 1
96
97
98     def test_person_cue_adds_features_when_enabled():
99         fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106             stats = _stats_calls(db)
107             assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108             assert all("net_crossings" in s for s in stats)
109             fake_pd.detections_per_frame.assert_called()         # the person detector WAS used
110

```

13. assistant (2026-06-13T19:15:41.374Z)

The test confirms the finding precisely. The only test exercising the person cue (``test_person_cue_adds_features_when_enabled``) hard-codes ``frame_height: 1000.0``. No test feeds ``frame_height=0.0``. And the test only asserts that the feature `*keys*` are present, not their `*values*` ? so even a zero-collapse would pass this test.

Now let me verify the finding's claim about the docstring warning ("the `detections_per_frame` docstring itself warns cv2 props are flaky per-codec"). Let me re-check the exact docstring. The docstring at lines 269-271 warns about cv2 `*frame_seek*` accuracy varying by codec, but does it warn about `*frame_width/frame_height props*` being flaky? Let me re-read carefully.

The docstring says: `"? cv2 frame_seek accuracy varies by codec (it lands near the requested keyframe then decodes forward)".` This is about frame_seek/POS_FRAMES accuracy, not about CAP_PROP_FRAME_HEIGHT being flaky. The finding slightly mischaracterizes the docstring ? the warning is about seek accuracy, not the dimension props specifically. However, the broader point stands: ``extract_action_windows_from_chunk`` (the sibling method) explicitly defends against ``frame_width <= 0`` with the comment "Sensible fallback" (line 163-164), establishing that the codebase author KNOWS cv2 dimension props can read non-positive ? so ``detections_per_frame`` reading them raw without a fallback is a genuine inconsistency.

Let me now verify the asymmetry's actual behavioral consequence by tracing through. With ``fh=0.0``:

- ``_foot_norm``: ``frame_height <= 0`` ? returns ``(0.0, 0.0)`` for every box (line 54-55).
- ``_center_norm``: same ? ``(0.0, 0.0)`` (line 61-62).
- ``mean_player_motion``: line 110 guards only on ``frame_width <= 0`` (not height), so it proceeds, but every ``_center_norm`` returns ``(0,0)``, so all displacement steps are ``0.0`` ? returns ``0.0``.
- ``shuttle_player_colocation``: line 171 guards ``frame_height <= 0`` ? returns ``0.0`` directly.

- `two\_sided\_motion` / `in\_court\_ratio` / `players\_in\_court`: depend on `person\_in\_court` ?
`\_foot\_norm` ? `(0,0)`. Under a court polygon, foot point `(0,0)` is likely outside ? counts collapse. With `court\_polygon=None`, `person\_in\_court` returns `True` regardless, so
`players\_in\_court`/`in\_court\_ratio` survive but `two\_sided\_motion`'s foot-based near/far split collapses (both coords 0 ? all "near").

So the finding's claim that these features "collapse to ~0 silently" is accurate, especially under a court polygon (which the fusion segmenter passes when configured, line 102-103).

Let me verify the finding's reproduction claim by actually running the math in Python to confirm
`mean\_player\_motion=0.0` and `shuttle\_player\_colocation=0.0` with fh=0 vs nonzero with fh=1000.

14. user (2026-06-13T19:15:46.831Z)

```
fh=0.0: mean_player_motion=0.000000 shuttle_player_colocation=0.000 two_sided=0.000
fh=1000.0: mean_player_motion=0.002000 shuttle_player_colocation=1.000 two_sided=0.000
--- with court_polygon ---
fh=0.0: players_in_court=0.0 in_court_ratio=0.0 two_sided=0.0
fh=1000.0: players_in_court=0.0 in_court_ratio=0.0 two_sided=0.0
```

15. assistant (2026-06-13T19:16:00.007Z)

The reproduction confirms the finding's core numbers exactly:

```
- `fh=0.0`: `mean_player_motion=0.000000`, `shuttle_player_colocation=0.000`
- `fh=1000.0`: `mean_player_motion=0.002000`, `shuttle_player_colocation=1.000`
```

These match the finding's stated "0.002 and 1.0 with fh=1000" vs "0.0 and 0.0" with fh=0.

Note: in my polygon example, players_in_court is 0 in both cases because that box (foot at y=40/1000=0.04) is outside my chosen polygon's y-range even at fh=1000 ? that's an artifact of my test geometry, not relevant to the finding. The key reproduced facts (`mean\_player\_motion` and `shuttle\_player\_colocation` collapsing) hold solidly.

Now let me assess severity honestly. Key questions for adversarial refutation:

1. ****Can cv2 actually return frame_height=0 in practice?**** The codebase's own sibling method `extract\_action\_windows\_from\_chunk` guards `frame\_width <= 0 ? 1280.0` with comment "Sensible fallback" (line 163-164), establishing the author's own belief that cv2 dimension props can read non-positive. So yes, this is a plausible-per-the-codebase scenario.
2. ****Is this default-OFF?**** Yes ? the person cue is opt-in (`cue\_flags.person`, default False), and the whole segmenter is default-OFF (not the default segmenter). This bounds the blast radius. But the finding accounts for this; it's about an untested asymmetry that "would ship unnoticed" within the person-cue path.
3. ****Is the asymmetry real, or is `fh = ... or 0.0` actually intentional / harmless?**** The `fw = det["frame\_width"]` or `frame\_width` line clearly intends a fallback to the substrate width when cv2 reports 0. The `fh = det["frame\_height"]` or `0.0` provides NO recovery ? `or 0.0` is a no-op (0.0 or 0.0 = 0.0; nonzero stays). So the asymmetry is real: width has a recovery path, height does not. There's no substrate `frame\_height` available to fall back to (the substrate probe `\_probe\_fps\_width` only returns fps+width), which is precisely WHY the author wrote `or 0.0` ? but that means height silently collapses everything rather than recovering.

Let me confirm the substrate only carries width, not height, to validate the "no analogous recovery available" framing.

16. user (2026-06-13T19:16:01.534Z)

```
58: def _probe_fps_width(video_path: str) -> tuple:
59:     """Return (fps, frame_width) for a video, with sensible fallbacks."""
83:         frame_width: float, video_path: str,
144:         fps, frame_width = self._probe_fps_width(video_path)
187:         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
204:         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
```

17. assistant (2026-06-13T19:16:06.692Z)

Confirmed: the substrate (`\_probe\_fps\_width`, line 58-59) returns `(fps, frame\_width)` only ? no `frame\_height`. So when cv2's `detections\_per\_frame` returns `frame\_height=0`, there is genuinely no substrate value to fall back to, which is exactly why the author wrote `or 0.0`. But that "fallback" silently zero-collapses the person features instead of either recovering height (e.g. from substrate aspect ratio) or flagging the features as unavailable. The asymmetry is real and structurally rooted.

Let me read the relevant lines of `\_probe\_fps\_width` to confirm it has the same kind of width fallback (showing the codebase pattern of defending dimension reads) and whether height is simply never probed.

18. user (2026-06-13T19:16:07.694Z)

```
56
57     @staticmethod
58     def _probe_fps_width(video_path: str) -> tuple:
59         """Return (fps, frame_width) for a video, with sensible fallbacks."""
60         cap = cv2.VideoCapture(video_path)
61         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
62         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
63         cap.release()
64         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
65
66     @staticmethod
67     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
68         """Provider-reported exchange count, else a duration-based estimate.
69
70         One canonical implementation ? the twins had drifted (wasb relied on
71         ``int(None)`` raising into the fallback; same result, by accident).
72         """
73         shot_count = segment.get("shuttle_exchange_count")
74         if shot_count is None:
75             return max(3, int(duration / 0.8))
76         try:
77             return int(shot_count)
78         except (ValueError, TypeError):
79             return max(3, int(duration / 0.8))
80
81     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
82     -----
83     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84                                frame_width: float, video_path: str,
85                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86         """Stats dict persisted into `candidate_segments.stats` for one window. The
87         BASE
```

19. assistant (2026-06-13T19:16:23.873Z)

This fully confirms the structural picture. `\_probe\_fps\_width` (line 64) defends width with `width if width > 0 else 1280.0` ? the codebase's consistent pattern of treating cv2 dimension reads as potentially non-positive. The fusion segmenter then propagates this defended width into `compute\_person\_features` via the `fw = det["frame\_width"]` or `frame\_width` fallback (line 99), but height has no such protection.

I now have everything needed to give a verdict. Let me summarize the verification:

****Confirmed facts:****

- Line 99-100 of `fusion\_hybrid.py`: `fw` has a substrate fallback, `fh` has only `or 0.0` (a no-op).
- `person\_detector.detections\_per\_frame` reads `CAP\_PROP\_FRAME\_HEIGHT` raw (line 283) with no guard ? and the sibling `extract\_action\_windows\_from\_chunk` proves the author knows these props can read ?0 (the 1280.0 width fallback, line 163-164).
- With `fh=0`, `compute\_person\_features` collapses: reproduced `mean\_player\_motion=0.0` and

``shuttle_player_colocation=0.0`` (vs `0.002 / 1.0` at `fh=1000`) ? matches the finding's numbers exactly.

- No test feeds ``frame_height=0``; the only person-cue test hard-codes ``1000.0`` and asserts key-presence only, not values ? so the asymmetry ships untested.

****Minor inaccuracy in the finding (does not undermine it):**** The finding says "the `detections_per_frame` docstring itself warns cv2 props are flaky per-codec." The docstring actually warns about frame-**seek** accuracy varying by codec, not the dimension-**prop** reads. The "props are flaky" support comes instead from the sibling method's explicit width fallback. This is a misattribution of the warning's exact subject, not a fabrication ? the underlying premise (cv2 dimension props can read 0, per this codebase's own defensive patterns) is solidly supported.

****Severity assessment:**** The finding labels this "high." I'd temper it to ****medium****: the consequence (silent feature collapse) is real and genuinely untested, and it would produce a hard-to-debug "everything looks like dead time" failure on a misbehaving codec. But the blast radius is bounded ? the entire segmenter is default-OFF (opt-in, never the default), the person cue within it is also default-OFF, and these features currently only feed **persisted** candidate stats for an offline learned-fusion experiment (P3), not a live gating decision yet (served Gate B gates on ``players_in_court < min_players``, which would also collapse ? but ``min_players`` defaults to 0, so no live gating by default). It's a latent data-quality/correctness bug in an opt-in experimental path, not a live production crash. That's solid "medium," not "high."

The finding is real.

20. user (2026-06-13T19:16:41.106Z)

Structured output provided successfully

21. assistant (2026-06-13T19:16:51.170Z)

I've completed the adversarial verification. The finding is ****REAL**** (severity: ****medium****, tempered from the claimed high).

Key verification points, all quoting actual code:

- ****The asymmetry is genuine**** ? ``backend/pipeline/segmenters/fusion_hybrid.py:99-100``: ``fw = det["frame_width"] or frame_width`` recovers width from the substrate, but ``fh = det["frame_height"] or 0.0`` is a no-op (0 stays 0). The substrate probe ``_probe_fps_width`` (``trajectory_hybrid.py:58-64``) returns only ``(fps, frame_width)`` ? no height to fall back to ? which is **why** the author wrote ``or 0.0``, but it silently zero-collapses rather than recovering.

- ****fh=0 is plausible per this codebase's own patterns**** ? the sibling method ``extract_action_windows_from_chunk`` (``person_detector.py:163-164``) and ``_probe_fps_width`` (line 64) both explicitly guard cv2 dimension reads against ?0, but ``detections_per_frame`` (lines 282-283) reads them raw.

- ****Reproduced the exact collapse****: `fh=0` ? ``mean_player_motion=0.0``, ``shuttle_player_colocation=0.0``; `fh=1000` ? ``0.002``, ``1.0``. Matches the finding's numbers precisely.

- ****Untested**** ? ``tests/test_fusion_hybrid.py:102`` hard-codes ``frame_height: 1000.0``, and the person-cue test only asserts feature **keys** exist (line 107), never values, so a zero-collapse passes green.

One minor misattribution in the finding (not load-bearing): it says the ``detections_per_frame`` docstring warns cv2 **props** are flaky per-codec, but the docstring actually warns about frame-**seek** accuracy. The "props can read 0" support comes instead from the sibling method's explicit fallback.

Severity is medium rather than high because the blast radius is bounded: the whole ``FusionSegmenter`` is opt-in (not the default), the person cue is default-OFF, and the features currently only feed **persisted** candidate stats for an offline P3 experiment ? no live gating decision is affected by default (``min_players`` defaults to 0). It's a latent data-quality bug in an experimental path, not a live crash.